

[← Back to API keys](#)

API reference

Authenticate with API key headers and an HMAC-SHA256 signature, then call the endpoints your key's scopes allow.

This guide shows how to sign requests, which endpoints each permission scope unlocks, and how to place a futures order.

Getting started

API keys let bots and external tools act on your account — read data, place futures orders, and move funds — through signed HTTP requests. Get going in three steps:

- Create an API key and select only the permission scopes you need.
- Copy the secret key shown once at creation — it can never be retrieved again.
- Sign every request with your secret as described below, and send the API key headers.

[Create an API key](#)

Base URLs

All API requests are made against the following base URLs:

- REST API — `https://apiv2.thetrue.trade.io`
- Futures trading — `https://apiv2.thetrue.trade.io/futures`

Required headers

Send these headers on every API-key-authenticated request:

- `X-API-Key` — Raw API key value.
- `X-Timestamp` — Current Unix timestamp in milliseconds (for example, `Date.now()` in JavaScript).
- `X-Signature` — Hex-encoded HMAC-SHA256 signature.
- `Content-Type` — Set to `application/json` for requests with a JSON body (POST, PATCH).



The server rejects requests that are older or newer than 30 seconds from server time.

How the signature is built

The server verifies this payload format:

```
{timestamp}{METHOD}{uri}
```

Rules

- `timestamp`: the same value you send in `X-Timestamp`.
- `METHOD`: uppercase HTTP method (GET, POST, PATCH, etc.).
- `uri`: the request URI path, including the query string when present (for example `/users/profile`, `/accounting/transfer?page=1`).
- `body`: never included — do not append the request body to the payload, even for POST and PATCH.

Then sign the payload

- Algorithm: HMAC-SHA256.
- Key: your API key secret.
- Output format: lowercase hex string.

Example: signed GET request

For a GET `/users/profile` request:

- Method: GET

- URI: /users/profile

The payload becomes:

```
{timestamp}GET/users/profile
```

Code examples

A complete, signed GET /users/profile request in each language.

Node.js Go Python

```
import hashlib
import hmac
import time

import requests

API_KEY = "your_api_key"
API_SECRET = "your_api_secret"
BASE_URL = "https://apiv2.thetrue.trade.io"

def sign(secret: str, timestamp: str, method: str, uri: str) -> str:
    payload = f"{timestamp}{method.upper()}{uri}"
    return hmac.new(secret.encode("utf-8"), payload.encode("utf-8"), hashlib.sha256).hexdigest()

def get_profile() -> None:
    method = "GET"
    uri = "/users/profile"
    timestamp = str(int(time.time() * 1000))
    signature = sign(API_SECRET, timestamp, method, uri)
```

```
headers = {
    "X-API-Key": API_KEY,
    "X-Timestamp": timestamp,
    "X-Signature": signature,
}

response = requests.get(f"{BASE_URL}{uri}", headers=headers, timeout=15)
response.raise_for_status()
print(response.json())

if __name__ == "__main__":
    get_profile()
```

Permission scopes

Each API key carries one or more permission scopes. A request is allowed only if the key holds the scope its route requires:

- `api-keys.readonly` — Read account, funding, and reference data — GET requests on `/users`, `/accounting`, `/currencies`, `/referrals`, and `/api-keys`.
- `api-keys.trade-futures` — Full access to every `/futures` route — market data, account reads, and placing or canceling orders.
- `api-keys.transfer` — Move funds between your accounts via `/accounting/transfer`.
- `api-keys.withdrawal` — Withdraw funds and cancel withdrawals via `/accounting/withdrawal`.



`trade-futures` is a single scope covering all of `/futures` — there is no read-only futures key. Any key that can read positions can also place and cancel orders.



If the route's scope is missing from the key's permissions, the request is rejected with 403.

Endpoints

Everything an API key can call, grouped by the scope it requires. Paths are relative to the REST base URL (futures routes sit under /futures).

Expand any endpoint to see its query parameters (GET) or request body (POST/PATCH) — and which fields are required versus optional.

api-keys.readonly

GET

/users/profile

Account & profile ^

No query parameters.

GET

/api-keys

List your API keys ^

Parameter	Type	Required	Description
page	integer	Optional	Page number for pagination.
limit	integer	Optional	Page size.
status	enum[]	Optional	Filter by key status.
sort	enum	Optional	createdAt, lastUsedAt, or expiresAt.
order	enum	Optional	asc or desc.

GET

/accounting/assets

Wallet & futures balances ^

No query parameters.

GET

/accounting/transactions

Transaction history ^

Parameter	Type	Required	Description
page	integer	Optional	Page number for pagination.
limit	integer	Optional	Page size.
startDate	ISO 8601	Optional	Start of the date range.
endDate	ISO 8601	Optional	End of the date range.

Parameter	Type	Required	Description
asset	string	Optional	Filter by asset symbol.
side	enum	Optional	Filter by transaction side.
type	enum	Optional	Filter by transaction type.
accountType	enum	Optional	Filter by account, e.g. funding or futures.
sort	enum	Optional	id or createdAt.
order	enum	Optional	asc or desc.

GET /accounting/deposits			Deposit history ^
Parameter	Type	Required	Description
offset	integer	Optional	Row offset for pagination.
limit	integer	Optional	Page size.
status	enum	Optional	Filter by deposit status.
currency	string	Optional	Filter by asset symbol.
startDate	ISO 8601	Optional	Start of the date range.
endDate	ISO 8601	Optional	End of the date range.
sort	enum	Optional	createdAt.
order	enum	Optional	asc or desc.

GET /accounting/withdrawals			Withdrawal history ^
Parameter	Type	Required	Description
page	integer	Optional	Page number for pagination.
limit	integer	Optional	Page size.

Parameter	Type	Required	Description
status	enum	Optional	Filter by withdrawal status.
currency	string	Optional	Filter by asset symbol.
from	ISO 8601	Optional	Start of the date range.
to	ISO 8601	Optional	End of the date range.

GET /accounting/transfers		Transfer history ^	
Parameter	Type	Required	Description
page	integer	Optional	Page number for pagination.
limit	integer	Optional	Page size.
from	ISO 8601	Optional	Start of the date range.
to	ISO 8601	Optional	End of the date range.
asset	string	Optional	Filter by asset symbol.
source	enum	Optional	Filter by source account.
destination	enum	Optional	Filter by destination account.

GET /accounting/deposit/currencies

Depositible currencies ^

No query parameters.

GET /accounting/withdrawal/currencies

Withdrawable currencies ^

No query parameters.

GET /currencies

Supported currencies ^

No query parameters.

POST /accounting/deposit/wallet		Get a deposit address ^	
---------------------------------	--	-------------------------	--

Field	Type	Required	Description
networkAssetId	integer	Required	Identifier of the network/asset combination.

GET /referrals/code
Your referral code ^

No query parameters.

api-keys.trade-futures


The trade-futures scope unlocks this entire group — reads and writes together.

Market data

GET /futures/markets
Markets & contract specs ^

No query parameters.

GET /futures/markets/stats
24h ticker statistics ^

No query parameters.

GET /futures/markets/orderbook
Order book ^

Parameter	Type	Required	Description
symbol	string	Required	Market symbol, e.g. BTCUSDT.
precision	string (decimal)	Optional	Price-level aggregation.

GET /futures/markets/trades
Recent market trades ^

Parameter	Type	Required	Description
symbol	string	Required	Market symbol, e.g. BTCUSDT.

GET /futures/markets/funding-history
Funding-rate history ^

Parameter	Type	Required	Description
symbol	string	Required	Market symbol, e.g. BTCUSDT.
page	integer	Required	Page number for pagination.
start	ISO 8601	Optional	Start of the date range.
end	ISO 8601	Optional	End of the date range.

GET /futures/udf/history		Candlesticks (klines) ^	
Parameter	Type	Required	Description
symbol	string	Required	Market symbol, e.g. BTCUSDT.
resolution	string	Required	Candle resolution, e.g. 15 or 1D.
from	integer (unix s)	Required	Range start, in seconds.
to	integer (unix s)	Required	Range end, in seconds.
countback	integer	Required	Number of bars to return.

GET /futures/quote-rates

Quote-asset to USD rates ^

No query parameters.

Account & positions

GET /futures/assets

Futures balances ^

No query parameters.

GET /futures/assets/pnl

PnL history ^

No query parameters.

GET /futures/positions		Open positions ^	
------------------------	--	------------------	--

Parameter	Type	Required	Description
symbol	string	Optional	Filter by market symbol.
active	boolean	Optional	Only open positions when true.
start	ISO 8601	Optional	Start of the date range.
end	ISO 8601	Optional	End of the date range.
page	integer	Optional	Page number for pagination.

GET /futures/orders			Open orders ^
Parameter	Type	Required	Description
active	boolean	Optional	Only working orders when true.
symbol	string	Optional	Filter by market symbol.
type	enum[]	Optional	Filter by order type (MARKET, LIMIT).
side	enum	Optional	Filter by side.
direction	enum	Optional	Filter by direction.
status	enum	Optional	Filter by order status.
start	ISO 8601	Optional	Start of the date range.
end	ISO 8601	Optional	End of the date range.
page	integer	Optional	Page number for pagination.

GET /futures/trades			Your trade history ^
Parameter	Type	Required	Description
symbol	string	Optional	Filter by market symbol.
side	enum	Optional	Filter by side.

Parameter	Type	Required	Description
direction	enum	Optional	Filter by direction.
start	ISO 8601	Optional	Start of the date range.
end	ISO 8601	Optional	End of the date range.
page	integer	Optional	Page number for pagination.

Trading

POST /futures/positions

Open a position ^

Documented in full under Create a futures order below — including required fields and examples.

POST /futures/positions/{id}/close

Close a position ^

Field	Type	Required	Description
orderType	enum	Required	Exit order type: MARKET or LIMIT.
price	string (decimal)	Optional	Exit price; required for a LIMIT close.
size	string (decimal)	Optional	Contract quantity to close.
notionalSize	string (decimal)	Optional	Notional value to close, in the quote asset.

POST /futures/positions/close-all

Close all positions ^

No request body.

PATCH /futures/positions/{id}/tpsl

Set or update TP/SL ^

Field	Type	Required	Description
takeProfit	string (decimal)	Optional	Take-profit trigger price.
takeProfitOrderType	enum	Optional	TAKE_PROFIT_MARKET or TAKE_PROFIT_LIMIT.

Field	Type	Required	Description
takeProfitExitPrice	string (decimal)	Optional	Exit price; required when the TP order type is TAKE_PROFIT_LIMIT.
takeProfitStrategy	enum	Optional	Trigger reference: LATEST_PRICE or MARK_PRICE.
stopLoss	string (decimal)	Optional	Stop-loss trigger price.
stopLossOrderType	enum	Optional	STOP_MARKET or STOP_LIMIT.
stopLossExitPrice	string (decimal)	Optional	Exit price; required when the SL order type is STOP_LIMIT.
stopLossStrategy	enum	Optional	Trigger reference: LATEST_PRICE or MARK_PRICE.
cancelTakeProfit	boolean	Optional	Set true to remove the take-profit.
cancelStopLoss	boolean	Optional	Set true to remove the stop-loss.

PATCH /futures/positions/{id}/add-margin Add margin to a position ^			
Field	Type	Required	Description
amount	string (decimal)	Required	Margin to add, in the quote asset.

DELETE /futures/orders/{id} Cancel an order ^			
No request body.			

POST /futures/orders/close-all Cancel all orders ^			
Field	Type	Required	Description
type	enum[]	Optional	Only cancel orders of these types (MARKET, LIMIT). Omit to cancel all.

POST /accounting/transfer

Transfer between accounts ^

Field	Type	Required	Description
amount	string (decimal)	Required	Amount to transfer.
asset	string	Required	Asset symbol, e.g. USDT.
source	enum	Required	Source account, e.g. funding or futures.
destination	enum	Required	Destination account, e.g. funding or futures.
remarks	string	Optional	Free-text note.

api-keys.withdrawal

POST /accounting/withdrawal

Create a withdrawal ^

Field	Type	Required	Description
walletAddress	string	Required	Destination wallet address, valid for the selected network.
networkAssetId	integer	Required	Identifier of the network/asset combination.
amount	string (decimal)	Required	Amount to withdraw.
remarks	string	Optional	Free-text note.

DELETE /accounting/withdrawals/{id}

Cancel a withdrawal ^

No request body.



The request body is never part of the signature — for POST and PATCH endpoints too, sign only the timestamp, method, and URI.

Create a futures order

Open a position by sending a POST to /futures/positions with the trade-futures scope. Provide exactly one of size, cost, or notionalSize.

- Endpoint — POST /futures/positions
- Required scope — api-keys.trade-futures

Market order (by collateral)

```
{
  "symbol": "BTCUSDT",
  "side": "LONG",
  "tradeType": "MARKET",
  "leverage": 10,
  "cost": "100",
  "takeProfit": "72000",
  "stopLoss": "61000",
  "walletType": "debit"
}
```

Limit order (by size)

```
{
  "symbol": "BTCUSDT",
  "side": "SHORT",
  "tradeType": "LIMIT",
  "leverage": 5,
  "price": "65000",
  "size": "0.01"
}
```

Key fields

- symbol — the futures market, e.g. BTCUSDT. Required.
- side — LONG or SHORT. Required.
- tradeType — MARKET or LIMIT. Required.
- leverage — integer within the market's allowed range. Required.
- size, cost, or notionalSize — provide exactly one. cost is collateral in the quote asset; notionalSize is position value; size is contract quantity.
- price — required for LIMIT orders; ignored for MARKET orders.
- takeProfit and stopLoss — optional trigger prices, each with optional order type, exit price, and trigger strategy.
- walletType — debit (default) or credit.



Sign the payload as timestamp + METHOD + URI — for this call that is the timestamp, then POST, then /futures/positions. The JSON body you send is not part of the signature.

Success response (201)

```
{
  "positionId": "1042"
}
```

Error response

```
{
  "errors": [
    { "field": "leverage", "message": "Leverage must be between 1 and 100 for t"
  ]
}
```

On success the gateway returns the new positionId. Errors come back as an errors array of message and field entries, with an HTTP status code.

Reading account data

Read endpoints return plain JSON. Sign them like any other request. A few common examples:

Balances — GET /accounting/assets

Returns one entry per asset and account type, with total and locked amounts.

```
[
  {
    "asset": "USDT",
    "accountType": "futures",
    "assetType": "debit",
    "balance": "1500.00",
    "lockedBalance": "200.00"
  }
]
```

Open positions — GET /futures/positions

Returns your currently open positions with size, leverage, mark price, and unrealized PnL.

```
[
  {
    "id": 1042,
    "symbol": "BTCUSDT",
    "side": "LONG",
    "status": "OPEN",
    "size": "0.015",
    "leverage": 10,
    "markPrice": "67250.5",
    "unrealizedPnL": "12.34"
  }
]
```

IP allowlist

If an allowlist is configured on the API key, requests from any other IP address are rejected.



Strongly recommended for any key with the withdrawal scope — see the security note below.

Rate limits

API usage is rate-limited per API key and per IP address. Exceeding the limit returns HTTP 429 — back off and retry.



Exact limits and quota response headers will be documented here once finalized.

Security best practices

Treat API keys like passwords. To limit your exposure:

- Store the secret key securely — it is shown only once and can never be recovered.
- Never commit keys to source control, logs, screenshots, chat tools, or public bots.
- Restrict each key to known IP addresses with the allowlist.
- Grant the fewest scopes a key needs, and set an expiry date.
- Rotate keys periodically and revoke any key you no longer use.



A key with the withdrawal scope skips the per-request 2FA and email codes the website requires — the key secret and IP allowlist are the only protection. Treat such keys as you would your funds, and always pair them with an IP allowlist.

Common mistakes

- Using a seconds timestamp instead of milliseconds.
- Signing a different URI than the one used in the actual request.

- Not uppercasing the HTTP method before signing.
- Appending the request body to the signature payload — the body is never signed.

Error reference

Errors return an errors array — each entry has a message and an optional field — alongside an HTTP status code. Common codes by area:

Authentication & permissions

Code	Message	Status
E_SECURITY_UNAUTHENTICATED	Unauthenticated	401
E_HTTP_UNAUTHORIZED_EXCEPTION	Unauthorized access	401
E_DATA_API_KEY_NOT_FOUND	API key not found	401
E_SECURITY_PERMISSION_DENIED	Permission denied	403
E_DATA_PERMISSIONS_NOT_FOUND	Permissions not found: {scopes}	403
E_BUSINESS_RULE_API_KEY_NOT_ACTIVE	Only active API keys can be used	403
E_SYSTEM_INTERNAL_SERVER_ERROR	Internal server error	500

Transfer

Code	Message	Status
E_VALIDATION_INVALID_AMOUNT	Invalid amount	400
E_BUSINESS_RULE_INSUFFICIENT_FUNDS	Insufficient funds	422
E_BUSINESS_RULE_CURRENCY_NOT_TRANSFERABLE	Currency is not transferable	422
E_DATA_TRANSFER_NOT_FOUND	Transfer not found	404

Withdrawal

Code	Message	Status
E_VALIDATION_INVALID_WALLET_ADDRESS	Invalid wallet address	400
E_VALIDATION_INVALID_NETWORK_ASSET	Invalid network asset	400
E_VALIDATION_WITHDRAWAL_AMOUNT_BELOW_MINIMUM	Withdrawal amount must be at least {min}	400
E_VALIDATION_WITHDRAWAL_AMOUNT_ABOVE_MAXIMUM	Withdrawal amount must not exceed {max}	400
E_BUSINESS_RULE_TOTP_NOT_ENABLED	Not allowed to withdraw without TOTP enabled	403
E_BUSINESS_RULE_INSUFFICIENT_FUNDS	Insufficient funds	422
E_BUSINESS_RULE_WITHDRAWAL_NOT_ALLOWED_FOR_KYC_LEVEL	Withdrawal is not allowed for your KYC level	403
E_BUSINESS_RULE_WITHDRAWAL_CANCEL_NOT_ALLOWED	Not allowed to cancel the withdrawal	422



Invalid signatures, expired timestamps (outside the 30-second window), and disallowed IPs are all rejected as 401 or 403 under E_SECURITY_UNAUTHENTICATED — check your signature and timestamp first.